Lecture 15: Hypertext Transfer Protocol (HTTP)

Objectives:

- Learn about the two main versions of the HTTP protocol, namely:
  - HTTP 1.0 (RFC 1945) and
  - HTTP 1.1 (RFC 2616)

1. Brief about Internet Protocols in general and HTTP in particular

Internet Protocols

Standard Internet protocols evolve through a community process called Request for Comments (RFC), through which all members of the Internet community can participate.

The discussion is supervised by a sub-committee (working group) of the Internet Engineering Task Force (IETF) in the related area.

IETF is a large open international community of network designers, operators, vendors, and researchers concerned with the evolution of the Internet architecture and the smooth operation of the Internet. It is open to any interested individual.

The detail of each internet protocol is documented and numbered with prefix RFC and is made available online.

HTTP Protocol

HTTP is an application-level protocol based on client-server architecture, designed for delivering hypermedia information on the web.

The design goals for HTTP are:

- *light protocol*: not consuming too many resources in client and server
- *fast protocol*: need to retrieve many widely distributed documents as fast as possible

According to the RFCs, HTTP is transport independent. However, in practice, HTTP servers use TCP protocol running on a default port 80.

The first version of the protocol was given version 0.9. However, the two versions that are now in operation are version 1.0 and version 1.1.

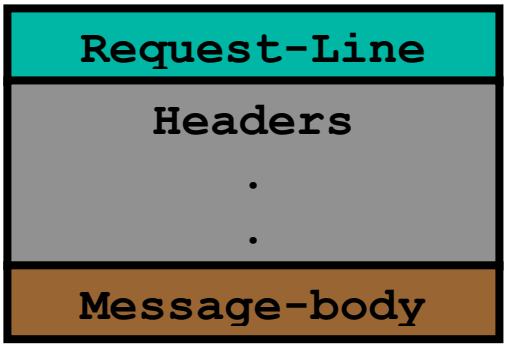
The following sections briefly discuss these two versions. For details about the protocols refer to the relevant links for the RFCs given above.

2. Overview of HTTP 1.0 (RFC 1945)

HTTP 1.0 is *stateless*: servers retain no information about past requests. Interaction between client and server has 4 phases:

- client connects to server
- client sends request to server
- server sends response to client
- server closes connection

2.1 Format of HTTP request



- each *client request* message has a format where each line ends with CRLF ("\r\n"):

```
method request-URI HTTP-version (request-line)
headers (0 or more lines)
<blank line> (CRLF)
message-body (only if a POST method)
```

- request methods (case sensitive):
  - **GET**: request document named by request-URI
  - **HEAD**: return only header information of request-URI (e.g., test for validity, recent modification)
  - **POST**: submit information to entity given by request-URI
  - other methods **(PUT, DELETE, etc.)**

- request-URI
  This specifies the full path of the resource relative to the server.
  eg:
  `/swe344/lectures/lecture1.html`

- HTTP-version
  This specifies the version of HTTP protocol that the client is able to handle. The values are: HTTP/1.0 or HTTP/1.1

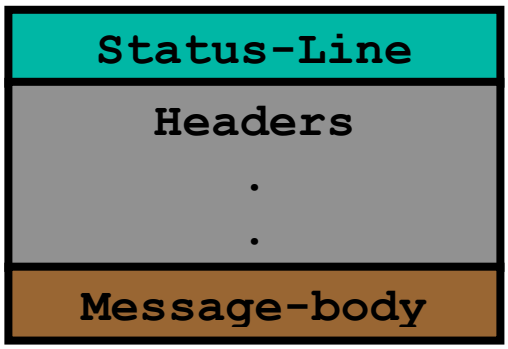
- Header Lines
  - Header lines provide information about the request or response, or about the object sent in the message body.
  - The header lines are in the form `"Header-Name: value"`, ending with CRLF.
  - The header name is not case-sensitive (but the value may be).
  - Any number of spaces or tabs may be between the ":" and the value.

  The following list some common header names:
  - `Accept` : what format is acceptable (client)
  - `User-Agent` : The client program (client)
  - `From` : email of the user (client)
  - `Content-Length` : length of the message (client/server)
  - `Content-Type:` type of the content (server)
  - `Date` : date sent (sever)
  - `Expires` : expiry date of the content (server)
  - `Last-Modified` : Last modification date (server)

- example of request:

```
GET /index.html HTTP/1.0
User-Agent: Mozilla/2.02Gold
Accept: image/gif, image/jpeg, */*
<blank line here>
```

2.2 Format of HTTP response



- each *server response* message has format:

```
HTTP-version, status-code, reason-phrase (status-line)
headers (0 or more lines)
<blank line> (CRLF)
message-body
```

- The status code is a three-digit integer, and the first digit identifies the general category of response:
    - **1xx** indicates an informational message only
    - **2xx** indicates success of some kind
    - **3xx** redirects the client to another URL
    - **4xx** indicates an error on the client's part
    - **5xx** indicates an error on the server's part

    The most common status codes are:
    200 OK : The request succeeded, and the resulting resource is returned in the message body.
    404 Not Found : The requested resource doesn't exist.
    301 Moved Permanently
    302 Moved Temporarily
    500 Server Error : An unexpected server error.

- example of response:

```
HTTP/1.0 200 OK
Date: Fri, 31 Dec 1999 23:59:59 GMT
Content-Type: text/html
Content-Length: 1354

<html>
<body>
<h1>Happy New Millennium!</h1>
(more file contents)
  .
  .
</body>
</html>
```

2.3 Testing HTTP Server

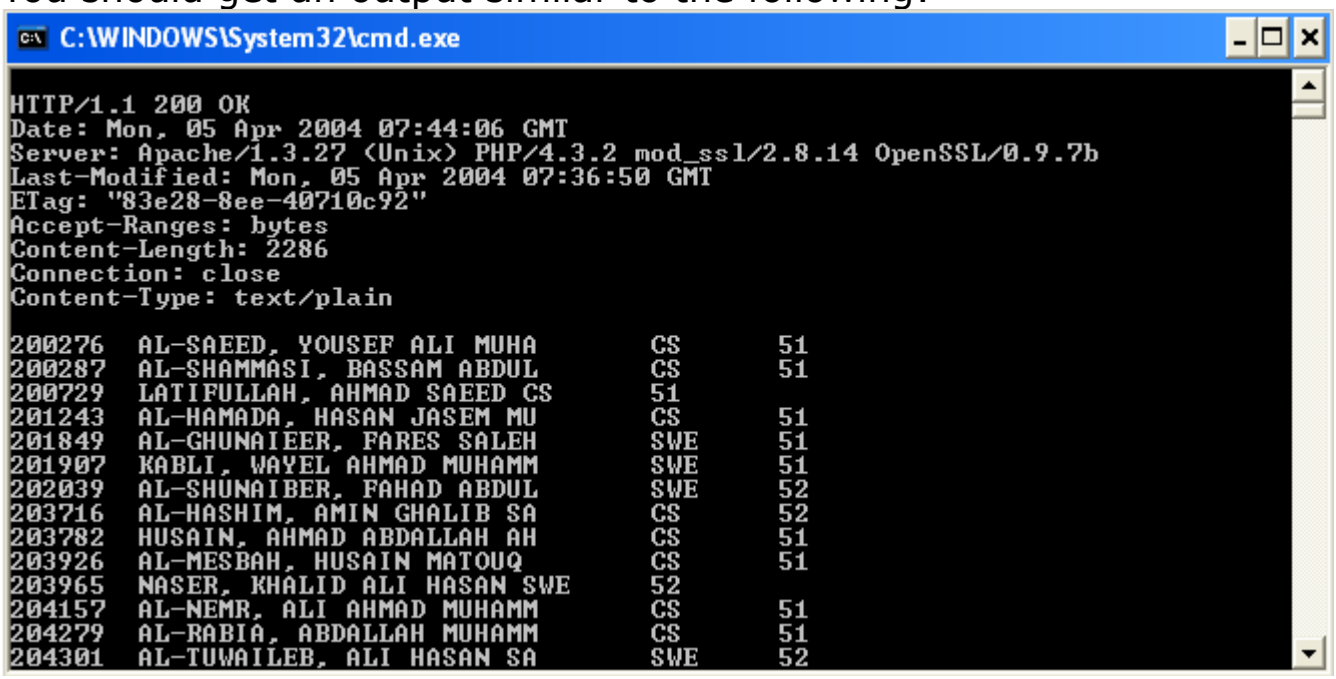
You can use TELNET to test a http server to visualize the above examples.

For example, to download the document:
http://www.ccse.kfupm.edu.sa/~bmghandi/swe344/swe344.txt

Enter the following commands on the DOS window:

```
telnet www.ccse.kfupm.edu.sa 80
GET /~bmghandi/swe344/swe344.txt HTTP/1.0
<blank Line>
```

You should get an output similar to the following:



Alternatively, you can write a TCP client using the TcpClient or the Socket class to retrieve the document.

```
void OnGetClicked(object sender, System.EventArgs e)
{
    String url = urlBox.Text;
    int doubleSlahIndex = url.IndexOf("//");
    if (doubleSlahIndex>0) {  //remove protocol part
        doubleSlahIndex+=2;
        url = url.Substring(doubleSlahIndex);
    }

    int pathIndex = url.IndexOf('/');
    string host = url.Substring(0, pathIndex);
    string path = url.Substring(pathIndex);

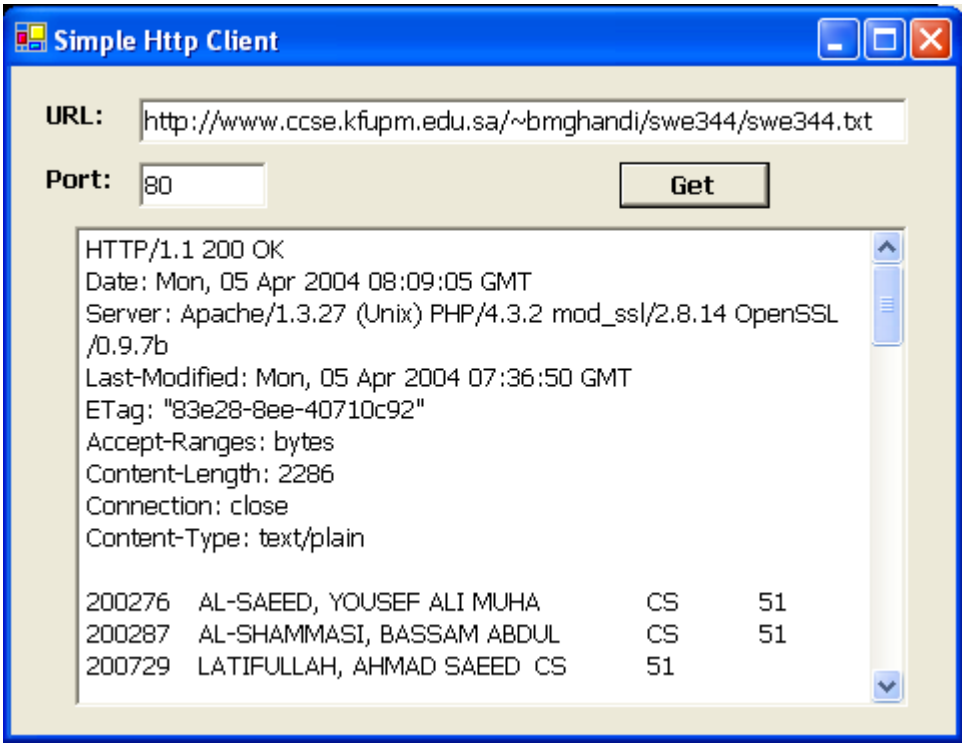
    int port = int.Parse(portBox.Text);

    client = new TcpClient(host, port);
    stream = client.GetStream();
    reader = new StreamReader(stream);
    writer = new StreamWriter(stream);

    String command = "GET "+path+ " HTTP/1.0"+"\r\n";
```

```
        writer.WriteLine(command);
        writer.Flush();

        string input;
        while((input = reader.ReadLine()) != null) {
            resultBox.Text += input + "";
        }
}
```



2. HTTP 1.1 (RFC 2616)

HTTP 1.1 has recently been defined to address new needs and overcome shortcomings of HTTP 1.0. Improvements include:

- Faster response, by allowing multiple transactions to take place over a single ***persistent*** *connection*.
- Faster response and great bandwidth savings, by adding cache support.
- Faster response for dynamically-generated pages, by supporting ***chunked encoding***, which allows a response to be sent before its total length is known.
- Efficient use of IP addresses, by allowing multiple domains to be served from a single IP address.

HTTP 1.1 requires a few extra things from both clients and servers

as explained below.

2.1 HTTP 1.1 Clients

To comply with HTTP 1.1, clients must:

- include the **Host** header with each request
- accept responses with *chunked* data
- either support *persistent connections*, or include the "**Connection: close**" header with each request
- handle the "**100 Continue**" response

2.1.1 Host Header

In HTTP 1.1, one server at one IP address can be *multi-homed*, i.e. the home of several Web domains. For example, "www.host1.com" and "www.host2.com" can live on the same server.

Several domains living on the same server is like several people sharing one phone: a caller knows who they're calling for, but whoever answers the phone doesn't. Thus, every HTTP request must specify which host name (and possibly port) the request is intended for, with the **Host** header.

A complete HTTP 1.1 request might be

```
GET /path/file.html HTTP/1.1
Host: www.host1.com:80
[blank line here]
```

Note: ":80" isn't required, since that's the default HTTP port.

Host is the only required header in an HTTP 1.1 request. *It's also the most urgently needed new feature in HTTP 1.1.* Without it, each host name requires a unique IP address, and we're quickly running out of IP addresses with the explosion of new domains.

2.1.2 Chunked Transfer-Encoding

If a server wants to start sending a response before knowing its total length (like with long script output), it might use the simple *chunked transfer-encoding*, which breaks the complete response into smaller chunks and sends them in series.

You can identify such a response because it contains the "**Transfer-Encoding: chunked**" header.

All HTTP 1.1 clients must be able to receive chunked messages. A chunked message body contains a series of *chunks*, followed by a line with "0" (zero), followed by optional footers (just like headers), and a blank line.

Each chunk consists of two parts:

- a line with the size of the chunk data, in hex, possibly followed by a semicolon and extra parameters, and ending with CRLF.
- the data itself, followed by CRLF.

So a chunked response might look like the following:

```
HTTP/1.0 200 OK
Date: Fri, 31 Dec 1999 23:59:59 GMT
Content-Type: text/plain
Transfer-Encoding: chunked

1a; ignore-stuff-here
abcdefghijklmnopqrstuvwxyz
10
1234567890abcdef
0
some-footer: some-value
another-footer: another-value
[blank line here]
```

Thus, the length of the text data is 42 bytes (1a + 10, in hex), and the data itself is **abcdefghijklmnopqrstuvwxyz1234567890abcdef**.

The footers should be treated like headers, as if they were at the top of the response.

2.1.3 Persistent Connections and the "Connection: close" Header

In HTTP 1.0 , TCP connections are closed after each request and response, so each resource to be retrieved requires its own connection.

Opening and closing TCP connections takes a substantial amount of CPU time, bandwidth, and memory.

In practice, most Web pages consist of several files on the same server, so much can be saved by allowing several requests and responses to be sent through a single ***persistent connection***.

Persistent connections are the default in HTTP 1.1, so nothing special is required to use them. Just open a connection and send several requests in series (called *pipelining*), and read the responses in the same order as the requests were sent.

If you do this, be very careful to read the correct length of each response, to separate them correctly.

If a client includes the "**Connection: close**" header in the request, then the connection will be closed after the corresponding response. **Use this if you don't support persistent connections**, or if you know a request will be the last on its connection.

Similarly, if a response contains this header, then the server will close the connection following that response, and the client shouldn't send any more requests through that connection.

A server might close the connection before all responses are sent, so a client must keep track of requests and resend them as needed.

When resending, don't pipeline the requests until you know the connection is persistent. Don't pipeline at all if you know the server won't support persistent connections (if it uses HTTP 1.0, based on a previous response).

2.1.4 The "100 Continue" Response

A HTTP 1.1 client sending a request to a server, the server might respond with an interim "**100 Continue**" response.

This means the server has received the first part of the request. HTTP 1.1 clients must handle the “100 Continue” response correctly (usually by just ignoring it).

The "**100 Continue**" response is structured like any HTTP

response, i.e. consists of a status line, optional headers, and a blank line. Unlike other responses, it is always followed by another complete, final response.

2.1.1 HTTP 1.1 Servers

To comply with HTTP 1.1, servers must:

- require the **Host:** header from HTTP 1.1 clients
- accept absolute URL's in a request
- accept requests with *chunked* data
- either support *persistent connections*, or include the "**Connection: close**" header with each response
- use the "**100 Continue**" response appropriately
- include the **Date:** header in each response
- handle requests with **If-Modified-Since:** or **If-Unmodified-Since:** headers
- support at least the GET and HEAD methods
- support HTTP 1.0 requests

2.1.2 Requiring the Host: Header

Servers are not allowed to tolerate HTTP 1.1 requests without the **Host** header. Instead, it must return a "**400 Bad Request**" response.

Example:

```
HTTP/1.1 400 Bad Request
Content-Type: text/html
Content-Length: 111

<html><body>
<h2>No Host: header received</h2>
HTTP 1.1 requests must include the Host: header.
</body></html>
```

This requirement applies *only* to clients using HTTP 1.1, not any future version of HTTP. See next section.

2.1.3 Accepting Absolute URL's

The **Host:** header is actually an interim solution to the problem of host identification. In future versions of HTTP, requests will use an absolute URL instead of a pathname, like:

```
GET http://www.somehost.com/path/file.html HTTP/1.2
```

To enable this protocol transition, HTTP 1.1 servers must accept this form of request, even though HTTP 1.1 clients won't send them.

2.1.4 Chunked Transfer-Encoding

Just as HTTP 1.1 clients must accept chunked responses, servers must accept chunked requests

Servers aren't required to generate chunked messages; they just have to be able to receive them.

2.1.5 Persistent Connections and the "Connection: close" Header

If an HTTP 1.1 client sends multiple requests through a single connection, the server should send responses back in the same order as the requests.

If a request includes the "**Connection: close**" header, that request is the final one for the connection and the server should close the connection after sending the response.

Also, the server should close an idle connection after some timeout period.

If server doesn't want to support persistent connections, it must include the "**Connection: close**" header in the response.

2.1.6 Using the "100 Continue" Response

When an HTTP 1.1 server receives the first line of an HTTP 1.1 (or later) request, it must respond with either "**100 Continue**" or an error.

If it sends the "**100 Continue**" response, it must also send another, final response, once the request has been processed.

The "**100 Continue**" response requires no headers, but must be followed by the usual blank line, like:

```
HTTP/1.1 100 Continue
```

```
[blank line here]
[another HTTP response will go here]
```

2.1.7 The Date: Header

Caching is an important improvement in HTTP 1.1, and can't work without timestamped responses.
Thus, servers must timestamp every response with a **Date:** header containing the current time, in the form

Date: Fri, 31 Dec 1999 23:59:59 GMT

All responses except those with 100-level status (but including error responses) must include the **Date:** header.

2.2.7 Handling Requests with `If-Modified-Since:` or `If-Unmodified-Since:` Headers

To avoid sending resources that don't need to be sent, thus saving bandwidth, HTTP 1.1 defines the **`If-Modified-Since:`** and **`If-Unmodified-Since:`** request headers.

The former says "only send the resource if it has changed since this date"; the latter says the opposite.

Clients aren't required to use them, but HTTP 1.1 servers are required to honor requests that do use them.

The **If-Modified-Since:** header is used with a GET request. If the requested resource has been modified since the given date, ignore the header and return the resource as you normal. Otherwise, return a "**304 Not Modified**" response, including the **Date:** header and no message body, like

```
HTTP/1.1 304 Not Modified
Date: Fri, 31 Dec 1999 23:59:59 GMT
[blank line here]
```

The **If-Unmodified-Since:** header is similar, but can be used with any method. If the requested resource has *not* been modified since the given date, ignore the header and return the resource as you normally would. Otherwise, return a "**412 Precondition Failed**" response, like:

```
HTTP/1.1 412 Precondition Failed
[blank line here]
```

2.2.8 Supporting the GET and HEAD methods

To comply with HTTP 1.1, a server must support at least the GET and HEAD methods.

If a client requests a method that is not supported, respond with "**501 Not Implemented**".

2.2.9 Supporting HTTP 1.0 Requests

To be compatible with older browsers, HTTP 1.1 servers must support HTTP 1.0 requests.

In particular, when a request uses HTTP 1.0 in the initial request line,

- don't require the **Host:** header, and
- don't send the **`"100 Continue"`** response.